Copy-on-Write Memory Demonstrator

Project Report

Team members

Name	Roll number
T. Jaswanth Reddy	2520030284
T. Karthik	2520030234
CH. Manikanta	2520030239

Abstract

When a Linux process calls `fork()`, the kernel does not immediately duplicate all of its memory. Parent and child initially share the same physical pages, and a private copy of a page is made only when one of them writes to it. This mechanism is called Copy-on-Write (COW). This project is an educational demonstrator for that behavior. A C program initializes a page-aligned private mapping, forks a child, lets the child write to it after a synchronization point, and shows that the parent's value is unchanged. While both processes are alive, the program samples `/proc/<pid>/smaps_rollup` to report memory accounting. An optional local Python and HTML dashboard launches the fixed compiled program and displays its real output. The report is careful about what the evidence can and cannot show: process-wide counters cannot identify one particular physical page, so they are presented as accounting snapshots rather than proof of a specific page copy.

1. Introduction

Creating a process is one of the most common operations in an operating system. A naive `fork()` would copy the entire address space of the parent, which is slow and wasteful, particularly when the child immediately calls `exec()` and discards the copy. Linux avoids this cost by sharing pages and deferring the copy until a write occurs. Although this idea is taught in every operating systems course, students rarely see it in action. This project makes the behavior visible through a small, real program and a simple dashboard.

2. Problem Statement

Students often learn that `fork()` uses Copy-on-Write without ever seeing a parent and child hold different values after a write. Printing two different values is also not enough on its own. Different variable values or different virtual addresses do not prove that physical memory was first shared and then copied. The problem is therefore to build a demonstration that:

- shows process isolation after a write in a shared-looking memory region,
- is deterministic, so every run follows the same sequence of phases, and
- uses real Linux memory accounting from `/proc` while being honest about what that accounting can prove.

3. Objectives

- Understand how the `fork()` system call creates a child process.
- Demonstrate that the child starts with the same data as the parent.
- Show that a write by the child does not change the parent's value.
- Observe memory accounting before and after the write using `/proc`.
- Understand how Copy-on-Write improves memory efficiency, since pages that are never written can remain shared.

4. Background

4.1 Virtual memory and pages

Each process sees its own virtual address space, divided into fixed-size pages (typically 4 KB). The kernel maps virtual pages to physical frames using page tables. Two processes can map different virtual addresses, or even the same virtual address, to the same physical frame.

4.2 fork()

`fork()` creates a child that is a copy of the parent. The child gets its own page tables, but they initially point to the same physical frames as the parent's.

4.3 Copy-on-Write

After `fork()`, the kernel marks writable private pages as read-only in both processes. When either process writes to such a page, the CPU raises a page fault. The kernel allocates a new frame, copies the contents, maps the new frame privately for the writing process, and restores write permission. The other process keeps the original frame. Pages that are never written stay shared, which is where the memory and time savings come from.

5. System Design

5.1 Components

Component	File	Role
C demonstrator	`src/cow_demo.c`	Page-aligned anonymous `mmap`, `fork`, pipes for synchronization, `waitpid`, and a reader for `/proc/<pid>/smaps_rollup`
Backend (optional)	`backend/app.py`	Localhost API built on Python's standard library that runs the fixed executable
Frontend (optional)	`frontend/`	Responsive HTML, CSS and JavaScript dashboard showing the program's output

5.2 Execution flow

The parent and child move through coordinated phases using pipes, so the observations always happen at known points:

1. The parent maps one or more page-aligned private pages and writes the starting value (default 42) into each page.
2. The parent calls `fork()`. The child reads the initial value and confirms it matches.
3. Both processes pause. The program samples `smaps_rollup` for the parent and the child (observation: after fork, before child write).
4. The child writes the new value (default 90) to one location on each page, which triggers Copy-on-Write for those pages.
5. The parent reads its own value after the child's write and finds it unchanged.
6. Both processes are sampled again (observation: after child write).

7. The child exits and the parent calls `waitpid()` to collect it and print the result.

5.3 Design decisions

- **Page-aligned anonymous mapping:** `mmap` gives a clean region of whole pages, so a write by the child affects exactly the pages being studied.
- **Pipes instead of sleeps:** pipes give deterministic phases, so observations do not depend on timing.
- **Honest dashboard:** the page drawing in the dashboard is explicitly conceptual. Live values and measurements come only from the C program, and without the backend the dashboard does not animate fake execution.

6. Implementation

6.1 The C program

The program accepts optional arguments for the number of pages, the parent's starting value and the child's write value. Values are bytes from 0 to 255 and must be different. It uses the following ideas:

- `mmap` with `MAP_PRIVATE | MAP_ANONYMOUS` to create the page-aligned region.
- `fork()` to create the child, with pipes used to hand control between the parent and the child at each phase.
- `waitpid()` so the parent finishes only after the child has exited.
- A small reader that opens `/proc/<pid>/smaps_rollup`, extracts the relevant fields and prints them. If a field is missing or the file is inaccessible, the program reports that rather than inventing a number.

The fields reported are `Rss`, `Pss`, `Shared_Clean`, `Shared_Dirty`, `Private_Clean`, `Private_Dirty`, and `Size` when the kernel provides it.

6.2 The backend

The optional backend is written with Python's standard library only, so no packages need to be installed. It was designed to be safe to run on a student machine:

- It binds to the loopback address only (`127.0.0.1`).
- It compiles and runs a fixed source file and a fixed executable path, with no shell involved.

- It accepts only a page count from 1 to 4096 and byte values from 0 to 255.
- It uses a run lock to prevent overlapping runs, a 20-second timeout, and a cap on output size.
- It serves the page and the API from one origin.

6.3 The frontend

The dashboard has fields for the parent's starting value and the child's write value, and a **Run Demonstration** button. After a run it shows the values and the `/proc` measurements returned by the C program.

7. Technologies Used

Technology	Purpose
C (GCC)	Implement the demonstration
Linux / Ubuntu (or WSL2)	Run the parent and child processes
`fork()`	Create the child process
`waitpid()`	Synchronize parent and child
`mmap`	Create a page-aligned private mapping
`/proc/<pid>/smaps_rollup`	Observe process memory accounting
Python 3.9+ (standard library)	Optional local backend
HTML, CSS, JavaScript	Optional dashboard

8. Requirements and How to Run

The project needs Ubuntu or another Linux system, or WSL2 on Windows, with GCC and Python 3.9 or later. No root access is required. Availability of `smaps_rollup` depends on the kernel and procfs configuration. On Windows, the source must not be compiled with native Windows GCC, because Windows has neither Linux `fork()` nor `/proc`.

Build and run the C program from the project root:

```
gcc -Wall -Wextra -O0 src/cow_demo.c -o cow_demo
./cow_demo
./cow_demo 8           # 8 pages
./cow_demo 1 17 203    # page count, parent value, child value
```

Start the optional dashboard:

```
python backend/app.py
```

Then open `http://127.0.0.1:5000` in a browser, set the two values and click **Run Demonstration**.

9. Interpreting the Memory Observations

The `smaps_rollup` fields describe the process as a whole:

- **Rss** is the amount of memory currently resident in RAM.
- **Pss** divides shared pages among the processes sharing them, so it gives a fairer per-process figure than Rss.
- **Shared_Clean / Shared_Dirty** are pages mapped by more than one process.
- **Private_Clean / Private_Dirty** are pages mapped by this process only.

These values are useful, but they have limits. They cover everything the process has mapped, including the C library and other runtime pages, and they can change with timing and kernel version. They are accounting snapshots, so they cannot prove which specific physical page was copied. Likewise, equal values or different virtual addresses do not prove physical sharing. The project uses these counters to support the explanation of Copy-on-Write, while the isolation of values is the direct, reliable result.

10. Testing

10.1 Procedure

For the standalone C test, compile and run the program with the commands in Section 8. For the full website test, start the backend, click **Run Demonstration**, and then check that the parent keeps 42, the child changes to 90, and the `/proc` measurements appear if the kernel makes them available.

10.2 Test cases

Test	What is checked	Expected result
Default run	Parent and child values with no arguments	Child reads 42, child writes 90, parent still reads 42
Multiple pages	`./cow_demo 8`	One location on each of 8 pages is modified; parent value unchanged
Custom values	`./cow_demo 1 17 203`	Parent keeps 17, child writes 203
Invalid input	Equal values, or values outside 0 to 255	Program or backend rejects the input
Observation phases	Two `/proc` samples are printed	One sample after fork and one after the child's write for both processes
Dashboard run	**Run Demonstration** with the backend running	Dashboard shows the real output of the C program
Backend limits	Page count above 4096, or overlapping runs	Request is rejected

The detailed expected and actual results are kept in `tests/test-cases.md`. Scenarios that have not been executed are marked as not run, and this report does not claim results for them.

11. Results

A run follows this shape (process IDs and memory counters vary between runs):

```
Copy-on-Write Memory Demonstrator
Parent PID: ...
Pages: 1 (... bytes)
Initial value at each page: 42
Child PID: ...
Child reads initial value: 42
[Observation: after fork, before child write]
...
Child modified 1 page location(s); value now: 90
Parent value after child write: 42 (unchanged)
[Observation: after child write]
...
Result: parent PID ... retains 42; child PID ... wrote 90.
```

The key result is process isolation: the child's write of 90 does not change the parent's 42. The two `/proc` observations show how the processes' memory accounting looks before and after the write, and they are consistent with pages being shared after `fork()` and made private when written. Specific measured numbers, along with dashboard screenshots, should be added from an actual Linux or WSL2 run before submission, because none have been captured in the environment where this report was prepared.

12. Limitations

- Native Windows does not provide `fork()` or `/proc`, so Linux, WSL2 or a Linux virtual machine is required.
- `smaps_rollup` values are process-wide, include unrelated mappings, and depend on kernel accounting, permissions and timing.
- The counters cannot confirm that one particular physical page was copied.
- The web interface runs a one-page demonstration per click. The backend validates page counts up to 4096 for direct API use.
- This is a teaching aid, not a benchmark or a page-table tracing tool.

13. Impact and Conclusion

Copy-on-Write is the reason `fork()` is cheap in Linux. Memory is copied only when needed, so process creation is faster and uses less RAM, and pages that are never modified remain shared. This project makes that idea concrete. It shows isolation with a

real `fork()`, deterministic phases and honest memory accounting, and it avoids overstating what the measurements prove. It is useful as a classroom demonstration and as a base for deeper study.

14. Future Work

- Use `/proc/<pid>/pagemap` (which normally needs elevated privileges) to compare physical frame numbers before and after the write, giving direct page-level evidence.
- Trace page faults with tools such as `perf` or `strace` and report fault counts around the write.
- Compare memory use for a large mapping when the child writes few pages versus all pages.
- Capture dashboard screenshots and record measured results from repeated runs.

15. Team Contributions

Team member	Contribution
T. Jaswanth Reddy	To be filled in before submission
T. Karthik	To be filled in before submission
CH. Manikanta	To be filled in before submission

References

- Linux manual pages: `fork(2)`, `mmap(2)`, `waitpid(2)` and `proc(5)` (for `smaps_rollup`).
- Project README and `tests/test-cases.md`.